



Universidad Veracruzana

Standard Template Library

Samuel Carreto Barrientos

Estructura de Datos

INTRODUCCION

Standard Template Library (STL)

La Standard Template Library (STL) es una colección de estructuras de datos y algoritmos genéricos escritos en C++. El diseño de la STL es el resultado de años de investigación llevada a cabo por Alexander Stepanov y Meng Lee de Hewlett-Packard, junto con David Musser del Rensselaer Polytechnic Institute.

Componentes de STL

La librería STL consta de cinco componentes distintos:

- **Algoritmos:** La STL provee una amplia variedad de algoritmos comunes, incluyendo funciones para ordenación, búsqueda y operaciones numéricas..
- **Iteradores:** Actúan como punteros para recorrer elementos de los contenedores.
- **Contenedores:** Son estructuras de datos diseñadas para contener y permitir la manipulación de otras estructuras de datos. Una estructura de datos se considera contenedora si puede albergar instancias de otros tipos de datos.
- **Funciones y objetos:** Varios algoritmos de la STL admiten que se les pase una función como argumento para adaptar su funcionalidad.
- **Adaptadores:** Modifican la interfaz de un componente para crear uno nuevo (por ejemplo, pilas o colas a partir de deques).

Tipos de Contenedores en STL

La STL incluye varios tipos de estructuras de datos contenedoras:

- **Contenedores Lineales:** Permiten el acceso a los elementos de manera secuencial y/o aleatoria. Ejemplos de contenedores lineales son los vectores y las colas de doble fin (deques).
- **Contenedores Asociativos:** Cada objeto almacenado está asociado a una clave. Esta clave facilita almacenar o recuperar objetos del contenedor de forma rápida. Ejemplos incluyen conjuntos (sets) y mapas (maps).
- **Contenedores Adaptados:** Permiten modificar la interfaz (métodos públicos y datos miembro) de un contenedor existente para crear uno nuevo. Las colas de prioridad son un ejemplo de contenedor adaptado.

Contenedores Lineales Específicos

VECTORES

Un vector es una clase genérica en STL que se utiliza para almacenar una colección de objetos del mismo tipo. Permiten almacenar múltiples elementos del mismo tipo y pueden crecer o encoger según sea necesario. Son indexados a partir de 0.

- Los elementos en un vector ocupan posiciones contiguas en memoria.
- Se puede acceder a los elementos de un vector utilizando un índice y el operador de indexación [].

Para utilizar el contenedor vector, es necesario incluir la cabecera:

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
```

CREAR UN VECTOR

Al ser una clase genérica, debes especificar el tipo de dato de los elementos que contendrá, usando la sintaxis:

```
6  /* vector<T> nombreVector;
7  donde T es el tipo de dato a usar */
8  vector<int> v;
9  vector<string> cadenas;
```

Si quieres agregar elementos al momento de declararlo, debes ponerlos entre llaves {} separados por comas:

```
11 // Vector llamado animales que contiene cadenas
12 vector<string> animales={"Gatos", "Pinguinos", "Osos", "Perros"};
```

ACCEDER A UN VECTOR

Puedes acceder a el elemento de un vector indicando el número del índice dentro de corchetes []

Por ejemplo, utilizando la cadena anterior:

```
12     vector<string> animales={"Gatos", "Pinguinos", "Osos", "Perros"};
13
14     // Escribe el primer elemento
15     cout<<animales[0]<<"\n"; // Escribe Gatos
16
17     // Escribe el segundo elemento
18     cout<<animales[1]<<"\n"; // Escribe Pinguinos
```

Una ventaja de usar esta biblioteca es que incluye funciones muy importantes como .front() para acceder el primer elemento y .back() para acceder al último elemento.

Ejemplo:

```
20     // Escribir el primer elemento
21     cout<<animales.front()<<"\n"; // Escribe Gatos
22
23     // Escribir el ultimo elemento
24     cout<<animales.back()<<"\n"; // Escribe Perros
```

Para acceder a un elemento en un índice específico también se puede utilizar la función .at().

Por ejemplo:

```
26     // Escribir el segundo elemento
27     cout<<animales.at(1)<<"\n"; // Escribe Pinguinos
28
29     // Escribir el tercer elemento
30     cout<<animales.at(2)<<"\n"; // Escribe Osos
```

La función .at() se utiliza más que los corchetes [] ya que, si ocurre un error, mandara un mensaje.

Por ejemplo, si el elemento esta fuera de rango:

```
32     // Al intentar acceder a un valor no existente mandara un mensaje de error
33     cout<<animales.at(6);
```

```
[0]: Gatos
[1]: Pinguinos
.front(): Gatos
.back(): Perros
.at(1): Pinguinos
.at(2): Osos
Valor fuera de rango: terminate called after throwing an instance of 'std::out_of_range'
what(): vector::_M_range_check: __n (which is 6) >= this->size() (which is 4)
```

CAMBIAR ELEMENTO EN UN VECTOR

Para cambiar el valor de un elemento específico, debes referirte al número de índice.

```
35 // Cambia el valor del tercer elemento
36 animales[2]="Serpientes";
37
38 cout<<"Valor cambiado: "<<animales.at(2)<<"\n"; // Ahora escribira Serpientes en lugar de Osos
```

AÑADIR ELEMENTOS A UN VECTOR

La diferencia entre vectores y arreglos es que en los vectores puedes agregar o quitar de manera dinámica elementos.

Para agregar un elemento se utiliza la función `.push_back()`, la cual agrega un elemento al final del vector.

```
39 // Agregar Patos
40 animales.push_back("Patos");
41
42 cout<<"Valor agregado: "<<animales.at(4)<<"\n";
```

ELIMINAR ELEMENTOS DE UN VECTOR

Para eliminar elementos de un vector se utiliza la función `.pop_back()`, esta elimina el último elemento del vector.

```
44 cars.pop_back(); // Esto eliminara el elemento Patos
```

TAMAÑO DEL VECTOR

Para saber cuántos elementos hay en un vector se utiliza la función `.size()`

```
46 cout<<"Tamano del vector: "<<animales.size()<<"\n"; // Escribira 4
```

VERIFICAR SI UN VECTOR ESTA VACIO

La función `.empty()` regresara 1 si el vector esta vacío, en el caso contrario regresara 0.

```
48     vector<string> colores;  
49     cout<<"Vector vacio: "<<colores.empty()<<"\n"; // Escribira 1  
50  
51     vector<int> num= {1, 2, 3, 4};  
52     cout<<"Vector no vacio: "<<num.empty()<<"\n"; // Escribira 0
```

RECORRER UN VECTOR

Puedes recorrer un vector utilizando un ciclo for combinado con la función `.size()`.

```
54     cout<<"\nVector recorrido\n";  
55     for(int i=0; i<animales.size(); i++){  
56         cout<<"Elemento "<<i+1<<": "<<animales[i]<<"\n";  
57     }
```

EJEMPLO

Todas estas funciones implementadas en un solo programa quedarían de la siguiente manera:

```
[0]: Gatos  
[1]: Pinguinos  
.front(): Gatos  
.back(): Perros  
.at(1): Pinguinos  
.at(2): Osos  
  
Valor cambiado: Serpientes  
  
Valor agregado: Patos  
  
Tamano del vector: 4  
  
Vector vacio: 1  
Vector no vacio: 0  
  
Vector recorrido  
Elemento 1: Gatos  
Elemento 2: Pinguinos  
Elemento 3: Serpientes  
Elemento 4: Perros  
  
Valor fuera de rango: terminate called after throwing an instance of 'std::out_of_range'  
what(): vector::_M_range_check: __n (which is 6) >= this->size() (which is 4)
```

COLAS DE DOBLE FIN

En una cola de doble fin (deque) los elementos se pueden agregar y eliminar desde el frente y desde atrás.

Para usar una cola de doble fin:

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
```

CREAR COLA DE DOBLE FIN

Para crear una cola de doble fin se utiliza la siguiente sintaxis

Deque<tipo> dequeNombre

Por ejemplo:

```
7  deque<int> num;
```

Y para agregar elemento al momento de la declaración:

```
9  deque<string> animales={"Gatos", "Pinguinos", "Osos", "Perros"};
```

ACCEDER A UNA DEQUE

Puedes acceder al elemento de una cola de doble fin refiriéndote al índice dentro de corchetes [].

Las colas de doble fin empiezan en 0, ósea que [0] es el primer elemento.

```
11  // Obtener el primer elemento
12  cout<<"[0]: "<<animales[0]<<"\n"; // Escribe Gatos
13
14  // Obtener el segundo elemento
15  cout<<"[1]: "<<animales[1]<<"\n"; // Escribe Pinguinos
```

También puedes acceder al primer o al ultimo elemento con las funciones `.front()` y `.back()`

Ejemplo:

```
17 // Escribir el primer elemento
18 cout<<"\n.front(): "<<animales.front()<<"\n"; // Escribe Gatos
19
20 // Escribir el ultimo elemento
21 cout<<"\n.back(): "<<animales.back()<<"\n"; // Escribe Perros
```

También puedes acceder a un elemento específico usando la función `.at()`.

Por ejemplo:

```
23 // Escribir el segundo elemento
24 cout<<"\n.at(1): "<<animales.at(1)<<"\n"; // Escribe Pingüinos
25
26 // Escribir el tercer elemento
27 cout<<"\n.at(2): "<<animales.at(2)<<"\n"; // Escribe Osos
```

CAMBIAR ELEMENTO EN UNA DEQUE

Para cambiar el valor de un elemento específico se utiliza el número del indicador.

```
29 // Cambia el valor del primer elemento
30 animales[0]="Buhos";
31 cout<<"\nValor cambiado: "<<animales.at(0); // Ahora escribira Buhos en lugar de Gatos
```

AGREGAR ELEMENTO

Para añadir elementos a una cola de doble fin se usa la función `.push_front()` para agregarlo al inicio y `.push_back()` para agregarlo al final.

Ejemplo

```
33 // Añade un elemento al inicio
34 animales.push_front("Conejos");
35 cout<<"\nElemento agregado al inicio: "<<animales.at(0)<<"\n";
36
37 // Añade un elemento al final
38 animales.push_back("Ranas");
39 cout<<"\nElemento agregado al final: "<<animales.at(5)<<"\n";
```


ELIMINAR ELEMENTOS DE UNA DEQUE

Para eliminar elementos de una deque se utilizan las funciones `.pop_front()` para eliminar un elemento del inicio y `.pop_back()` para eliminar un elemento del final.

```
41 // Elimina el primer elemento
42 animales.pop_front(); // Elimina Conejos
43
44 // Elimina el ultimo elemento
45 animales.pop_back(); // Elimina Ranas
```

TAMAÑO DEQUE

Para saber cuántos elementos hay en una cola de doble fila se utiliza la función `.size()`

Ejemplo

```
47 cout<<"\nTamano de la Cola: "<<animales.size()<<"\n"; // Escribe 4
```

VERIFICAR SI LA DEQUE ESTA VACIA

Se usa la función `.empty()`, regresara 1 si la cola esta vacia y 0 si es el caso contrario.

Ejemplo

```
49 deque<float> calif;
50 cout<<"\nCola vacia: "<<calif.empty()<<"\n"; // Escribe 1
51
52 deque<string> comidas= {"Pizza", "Chilaquiles"};
53 cout<<"Cola no vacia: "<<comidas.empty()<<"\n"; // Escribe 0
```

RECORRER UNA DEQUE

Al igual que con el vector, la deque se puede recorrer con un ciclo for junto a la función `.size()`:

```
55 cout<<"\nCola recorrida: \n";
56 for(int i=0; i<animales.size(); i++) {
57     cout<<"Elemento "<<i+1<<": "<<animales[i]<<"\n";
58 }
```

EJEMPLO

Al momento de implementar todas las funciones anteriores en un solo programa queda de la siguiente manera:

```
[0]: Gatos
[1]: Pinguinos

.front(): Gatos
.back: Perros

.at(1): Pinguinos
.at(2): Osos

Valor cambiado: Buhos

Elemento agregado al inicio: Conejos
Elemento agregado al final: Ranas

Tamano de la Cola: 4

Cola vacia: 1
Cola no vacia: 0

Cola recorrida:
Elemento 1: Buhos
Elemento 2: Pinguinos
Elemento 3: Osos
Elemento 4: Perros
```

Contenedores Asociativos

CONJUNTOS (set)

Un conjunto guarda elementos únicos:

- Se ordenan de manera automática en orden ascendente
- Cada uno es único, esto significa que los duplicados son totalmente ignorados.
- Se pueden agregar y eliminar elementos, pero el valor de un elemento ya existente no puede ser cambiado.

Para utilizar los conjuntos se debe escribir

```
1  #include<iostream>
2  #include<set>
3  using namespace std;
```

CREAR UN CONJUNTO

Para crear un conjunto se utiliza la sintaxis `set<tipo> nombreSet`

Por ejemplo:

```
7  set<int> num;
```

Si se quieren agregar elementos al momento de declararlos se puede hacer de la siguiente manera:

```
9  set<string> animales={"Gatos", "Pinguinos", "Osos", "Perros"};
```

```
11 // Escribir elementos de un conjunto
12 for(string animal : animales){ //Se utiliza un for-each
13     cout<<animal<<"\n";
14 }
```

La salida seria:

```
Gatos
Osos
Perros
Pinguinos
```

Los elementos en un mapa se acomodan de manera automática, en este caso al ser una cadena, en orden alfabético, si fueran números se acomodarían de menor a mayor.

```
18     set<int> numeros = {1, 7, 3, 2, 5, 9};
19
20     // Escribir elementos de un conjunto
21     for (int nume : numeros){
22         cout<<nume<<"\n";
23     }
```

La salida seria:

```
1
2
3
5
7
9
```

ORDENAR UN CONJUNTO DE MANERA DESCENDIENTE

Si quieres ordenar el conjunto de manera descendiente se utilizará el functor `greater<tipo>` dentro de la creación del conjunto.

Por ejemplo:

```
27     set<int, greater<int>> calif = {5, 8, 4, 2, 6, 9};
28
29     // Imprimir los números
30     for (int cal : calif) {
31         cout<<cal<<"\n";
32     }
```

La salida seria:

```
9
8
6
5
4
2
```

AÑADIR ELEMENTOS

Para añadir elementos a un conjunto se utiliza la función `.insert()`

Ejemplo:

```
34 // Añadir nuevos elementos
35 animales.insert("Serpientes");
36 animales.insert("Conejos");
37
38 cout<<"\nConjunto con los nuevos elementos: \n";
39 for(string animal : animales){
40     cout<<animal<<"\n";
41 }
```

ELIMINAR ELEMENTOS

Para eliminar un elemento específico se utiliza la función `.erase()`

Por ejemplo:

```
43 // Eliminar elementos
44 animales.erase("Osos");
45 animales.erase("Perros");
46
47 cout<<"\nConjunto sin los elementos eliminados: \n";
48 for(string animal : animales){
49     cout<<animal<<"\n";
50 }
```

Para eliminar todos los elementos de un conjunto se utiliza la función `.clear()`

Por ejemplo:

```
52 // Eliminar a todos los elementos
53 animales.clear();
```

ENCONTRAR EL TAMAÑO DE UN CONJUNTO

Para encontrar el tamaño de un conjunto se utiliza la función `.size()`

```
55 cout<<"\n"<<"Tamaño animales despues de borrarlo: "<<animales.size()<<"\n"; // Escribe 0
56 cout<<"Tamaño numeros: "<<numeros.size()<<"\n"; // Escribe 6
```

VERIFICAR SI UN CONJUNTO ESTA VACIO

Se utiliza la función `.empty()` para saber si se está vacío o no. Regresa 1 si esta vacio y 0 si no lo está.

```
58 cout<<"\nConjunto vacio: "<<animales.empty()<<"\n"; // Escribe 1
59 cout<<"Conjunto no vacio: "<<calif.empty()<<"\n"; // Escribe 0
```

EJEMPLO

Al momento de implementar estas funciones en el mismo programa, quedaría de la siguiente manera:

Gatos
Osos
Perros
Pinguinos

1
2
3
5
7
9

9
8
6
5
4
2

Conjunto con los nuevos elementos:

Conejos
Gatos
Osos
Perros
Pinguinos
Serpientes

Conjunto sin los elementos eliminados:

Conejos
Gatos
Pinguinos
Serpientes

Tamano animales despues de borrarlo: 0

Tamano numeros: 6

Conjunto vacio: 1

Conjunto no vacio: 0

MAPAS

Un mapa permite asociar en orden una lista de parejas a valores o claves.

Las características de un mapa son:

- Se puede acceder mediante claves, los valores pueden ser iguales, pero cada clave es única
- Ordena automáticamente en orden ascendente

Para usar los mapas:

```
1  #include<iostream>
2  #include<map>
3  using namespace std;
```

CREAR UN MAPA

Para crear un mapa se tiene que indicar el tipo de dato de la clave y del valor.

Por ejemplo:

```
7  // Crear un mapa que contenga cadenas
8  map<string, int> animales;
```

Si se quieren agregar elementos al momento de declararse:

```
10 // Crear un mapa con personas y sus edades
11 map<string, int> personas= { {"Taylor", 35}, {"Olivia", 22}, {"Sabrina", 26} };
```

ACCEDER A UN MAPA

Para acceder a un elemento en un mapa, tienes que referirte a su clave, a diferencia de los vectores, no puedes utilizar índices.

Ejemplo:

```
13 // Escribir el valor asociado con la clave Taylor
14 cout<<"Taylor tiene: "<<personas["Taylor"]<<"\n";
15
16 // Escribir el valor asociado con la clave Olivia
17 cout<<"Olivia tiene: "<<personas["Olivia"]<<"\n";
```


También puedes acceder a los elementos con la función .at()

Ejemplo:

```
19 // Obtiene el valor asociado a la clave Olivia
20 cout<<".at(Olivia): Olivia tiene: "<<personas.at("Olivia")<<"\n";
21
22 // Obtiene el valor asociada a la clave Sabrina
23 cout<<".at(Sabrina): Sabrina tiene: "<<personas.at("Sabrina")<<"\n";
```

CAMBIAR VALORES

Para cambiar los valores en un mapa se pueden usar las claves o la función .at()

```
25 // Cambiar valor de Taylor de 35 a 50
26 personas["Taylor"]=50;
27 cout<<"\nTaylor tiene: "<<personas["Taylor"]<<" despues de modificar el valor\n";
```

AÑADIR ELEMENTOS

Para añadir elementos a un mapa se pueden usar corchetes [], o la función .insert()

```
29 // Añadir elementos [ ]
30 personas["Chappell"]=27;
31 personas["Laufey"]=26;
32 // Añadir elementos .insert()
33 personas.insert({"Doechii", 26});
34 personas.insert({"Ariana", 31});
```

ELIMINAR ELEMENTOS

Para eliminar un elemento de un mapa se utiliza la función .erase():

```
36 // Eliminar elemento por clave
37 personas.erase("Doechii");
```

Para eliminar todos los elementos de un mapa, se utiliza la función .clear()

```
39 map<string, int> personas2={ {"Billie", 32}, {"Rachel", 45}, {"Lorde", 29} };
40 // Eliminar todos los elementos
41 personas2.clear();
```

ENCONTRAR EL TAMAÑO DE UN MAPA

Para encontrar cuantos elementos tiene un mapa se utiliza la función .size()

```
43 cout<<personas.size(); // Escribira 6
```

VERIFICAR SI UN MAPA ESTA VACIO

Se utiliza la función `.empty()`:

Esta función regresara 1 si esta vacio y 0 en el caso contrario:

```
45 cout<<"\nMapa vacio: "<<personas2.empty()<<"\n"; // Escribira 1
46
47 cout<<"Mapa no vacio: "<<personas.empty()<<"\n"; // Escribira 0
```

Tambien puedes verificar si un elemento especifico existe, utilizando la función `.count(clave)`.

Si regresa 1, el elemento existe, regresara 0 si no existe:

```
49 cout<<"\nBuscar la clave Ariana: "<<personas.count("Ariana")<<"\n"; // Escribira 1
50 cout<<"Buscar la clave Doechi: "<<personas.count("Doechi")<<"\n"; // Escribira 0
```

RECORRER UN MAPA

Puedes recorrer un mapa utilizando un ciclo `for-each`, pero tienes que tener unas cosas en cuenta:

- Debes utilizar el `auto` keyword dentro del ciclo, esto permitirá que el compilador determine automáticamente el tipo de dato correcto a utilizar.
- Ya que el mapa tiene claves y valores, tendrás que escribir `.first` para acceder a las claves y `.second` para acceder al valor.

Example

```
52 cout<<"\nMapa recorrido:\n";
53 for(auto person : personas){
54     cout<<person.first<<" tiene: "<<person.second<<"\n";
55 }
```

La salida será:

```
Mapa recorrido:
Ariana tiene: 31
Chappell tiene: 27
Laufey tiene: 26
Olivia tiene: 22
Sabrina tiene: 26
Taylor tiene: 50
```

Si quieres voltear el orden se puede usar el functor `greater<tipo>`

```
57     cout<<"\nMapa recorrido con greater<>:\n";
58     map<string, int, greater<string>> personas3={ {"Taylor", 35}, {"Olivia", 22},
59     {"Sabrina", 26}, {"Chappell", 27}, {"Laufey", 26}, {"Ariana", 31} };
60     for(auto per : personas3){
61         cout<<per.first<<" tiene: "<<per.second<<"\n";
62     }
```

La salida seria:

```
Mapa recorrido con greater<>:
Taylor tiene: 35
Sabrina tiene: 26
Olivia tiene: 22
Laufey tiene: 26
Chappell tiene: 27
Ariana tiene: 31
```

EJEMPLO

Al implementar todo en un solo código, el resultado quedaria de la siguiente manera:

```
[Taylor]: Taylor tiene: 35
[Olivia]: Olivia tiene: 22
.at(Olivia): Olivia tiene: 22
.at(Sabrina): Sabrina tiene: 26

Taylor tiene: 50 despues de modificar el valor

Tamanio del mapa: 6

Mapa vacio: 1
Mapa no vacio: 0

Buscar la clave Ariana: 1
Buscar la clave Doechi: 0

Mapa recorrido:
Ariana tiene: 31
Chappell tiene: 27
Laufey tiene: 26
Olivia tiene: 22
Sabrina tiene: 26
Taylor tiene: 50

Mapa recorrido con greater<>:
Taylor tiene: 35
Sabrina tiene: 26
Olivia tiene: 22
Laufey tiene: 26
Chappell tiene: 27
Ariana tiene: 31
```

Contenedores Adaptados

COLAS DE PRIORIDAD

Las colas de prioridad son un tipo de colas en las que cada elemento está asociado a un valor de prioridad.

En estas colas los primeros elementos en salir son los que tienen la mayor prioridad

Para empezar a usar las colas de prioridad se debe incluir:

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
```

CREAR UNA COLA DE PRIORIDAD

Para crear una cola de prioridad se utiliza la siguiente sintaxis:

```
priority_queue<tipo> nombreCola;
```

Donde tipo indica el tipo de dato:

```
7  // Crear una cola de prioridad para un int
8  priority_queue<int> cpInt;
9  // Crear una cola de prioridad para un string
10 priority_queue<string> cpString;
```

AÑADIR ELEMENTO

Para insertar un elemento se utiliza la función `.push()`

Ejemplo:

```
12     num.push(1);
13     num.push(20);
14     num.push(7);
15
16     cout<<"Primer lista:\n";
17     while (!num.empty()){
18         cout<<num.top()<<"\n";
19         num.pop();
20     }
```

Cola de prioridad:

```
Primer cola:
20
7
1
```

ELIMINAR ELEMENTOS

Para eliminar un elemento se utiliza la función `.pop()`, esta eliminara el elemento de mayor prioridad.

```
26     num.pop();
```

Cola inicial:

```
22     num.push(28);
23     num.push(21);
24     num.push(5);
```

Cola con el primer elemento eliminado:

```
Segunda cola:  
21  
5
```

ACCEDER A UN ELEMENTO

Para acceder a un elemento se usa la función `.top()`, obtiene el elemento con mayor prioridad:

```
34 num.push(21);  
35 num.push(5);  
36 num.push(28);  
37  
38 // Obtiene el elemento con mayor prioridad  
39 int top=num.top();  
40 cout<<"\nElemento mayor: "<<top<<"\n"; // Escribira 28
```

ENCONTRAR EL TAMAÑO DE UNA COLA DE PRIORIDAD

Se utiliza la función `.size()` para saber el numero de elementos que contiene.

Ejemplo:

```
42 int size=num.size();  
43 cout<<"\nTamaño de la cola: "<<size; // Escribira 3
```

VERIFICAR SI ESTA VACIA

Se utiliza la función `.empty()`, regresara 1 si esta vacia y en el caso contrario regresara 0.

Por ejemplo:

```
45 cout<<"\nCola no vacia: "<<num.empty()<<"\n"; //Regresa 0  
46 num.pop();  
47 num.pop();  
48 num.pop();  
49 cout<<"Cola vacia: "<<num.empty()<<"\n"; //Regresa 1
```

EJEMPLO

Implementando estas funciones queda de la siguiente manera:

```
Primer cola:  
20  
7  
1  
  
Segunda cola:  
21  
5  
  
Elemento mayor: 28  
  
Tamano de la cola: 3  
  
Cola no vacia: 0  
Cola vacia: 1
```


FUENTES

C++ Deque. (s/f). W3schools.com. Recuperado el 23 de mayo de 2025, de https://www.w3schools.com/cpp/cpp_deque.asp

C++ maps. (s/f). W3schools.com. Recuperado el 23 de mayo de 2025, de https://www.w3schools.com/cpp/cpp_maps.asp

C++ priority queue. (s/f). Programiz.com. Recuperado el 23 de mayo de 2025, de <https://www.programiz.com/cpp-programming/priority-queue>

C++ sets. (s/f). W3schools.com. Recuperado el 23 de mayo de 2025, de https://www.w3schools.com/cpp/cpp_sets.asp

C++ vectors. (s/f). W3schools.com. Recuperado el 23 de mayo de 2025, de https://www.w3schools.com/cpp/cpp_vectors.asp

Ejemplos — Fundamentos de Programación en C++. (s/f). Uva.es. Recuperado el 23 de mayo de 2025, de https://www2.eii.uva.es/fund_inf/cpp/temas/9_vectores/ejemplos.html

La Standard Template Library (STL). (s/f). Ugr.es. Recuperado el 23 de mayo de 2025, de <https://ccia.ugr.es/~jfv/ed1/c++/cdrom4/paginaWeb/stl.htm>

